

Finite Automata

Deterministic, Nondeterministic, and State Minimization

Computer Science Theory

Slide Set 2

Table of Contents

- 1 Deterministic Finite Automata (DFA)
- 2 Designing DFAs
- 3 Nondeterministic Finite Automata (NFA)
- 4 Equivalence of DFA and NFA
- 5 DFA Minimization (State Reduction)
- 6 Exercises and Review

By the end of this slide set, you should be able to:

- define DFAs and ε -NFAs formally and trace their computations;
- design finite automata whose states record exactly the needed information about an input prefix;
- convert an ε -NFA to an equivalent DFA using subset construction;
- identify reachable, distinguishable, and equivalent DFA states; and
- minimize a DFA using the table-filling (marking) algorithm.

Part 1: Intuitive Introduction to DFAs

Self-Study Header: The Automatic Door Controller

Finite automata are excellent models for computers with an extremely limited amount of memory.

Part 1: Intuitive Introduction to DFAs

Self-Study Header: The Automatic Door Controller

Finite automata are excellent models for computers with an extremely limited amount of memory.

Example: Automatic Door Controller

The controller has states **OPEN** and **CLOSED**. Its input records whether a person is detected at the front, rear, both sides, or neither side.

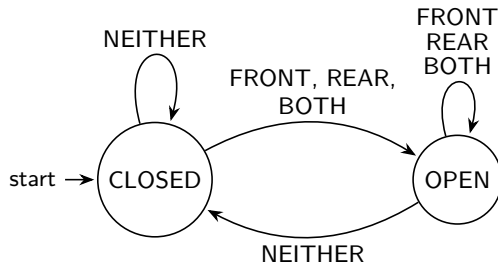
Part 1: Intuitive Introduction to DFAs

Self-Study Header: The Automatic Door Controller

Finite automata are excellent models for computers with an extremely limited amount of memory.

Example: Automatic Door Controller

The controller has states **OPEN** and **CLOSED**. Its input records whether a person is detected at the front, rear, both sides, or neither side.



Formal Definition of a DFA

To resolve any ambiguities, we require a mathematically precise definition.

Formal Definition of a DFA

To resolve any ambiguities, we require a mathematically precise definition.

The 5-Tuple Definition

A deterministic finite automaton is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where:

- 1 Q is a finite set called the states,
- 2 Σ is a finite set called the alphabet,
- 3 $\delta : Q \times \Sigma \rightarrow Q$ is the transition function,
- 4 $q_0 \in Q$ is the start state, and
- 5 $F \subseteq Q$ is the set of accept states.

Formal Definition of a DFA

To resolve any ambiguities, we require a mathematically precise definition.

The 5-Tuple Definition

A deterministic finite automaton is a 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, where:

- 1 Q is a finite set called the states,
- 2 Σ is a finite set called the alphabet,
- 3 $\delta : Q \times \Sigma \rightarrow Q$ is the transition function,
- 4 $q_0 \in Q$ is the start state, and
- 5 $F \subseteq Q$ is the set of accept states.

Crucial detail: δ is total. For every $q \in Q$ and $a \in \Sigma$, exactly one transition $\delta(q, a)$ is defined.

Extended Transition Function

The transition function δ strictly handles single symbols. To describe the processing of entire strings, it is convenient to introduce the extended transition function

$$\delta^* : Q \times \Sigma^* \rightarrow Q.$$

Extended Transition Function

The transition function δ strictly handles single symbols. To describe the processing of entire strings, it is convenient to introduce the extended transition function $\delta^* : Q \times \Sigma^* \rightarrow Q$.

Recursive Definition of δ^*

We define δ^* recursively as follows:

- 1 $\delta^*(q, \varepsilon) = q$.
- 2 $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$ for all $w \in \Sigma^*$ and $a \in \Sigma$.

Extended Transition Function

The transition function δ strictly handles single symbols. To describe the processing of entire strings, it is convenient to introduce the extended transition function $\delta^* : Q \times \Sigma^* \rightarrow Q$.

Recursive Definition of δ^*

We define δ^* recursively as follows:

- ① $\delta^*(q, \varepsilon) = q$.
- ② $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$ for all $w \in \Sigma^*$ and $a \in \Sigma$.

Language recognized by M :

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \in F\}.$$

A language is **regular** if it is recognized by some DFA.

Designing DFAs: Case Study 1

Goal: Construct a DFA accepting all strings over $\{0, 1\}$ with an **odd number of 1s**.

Designing DFAs: Case Study 1

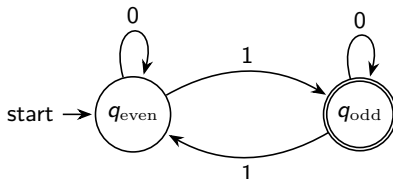
Goal: Construct a DFA accepting all strings over $\{0, 1\}$ with an **odd number of 1s**.

Thought Process: We don't need to remember the entire string. We only need to remember whether the number of 1s seen so far is even or odd.

Designing DFAs: Case Study 1

Goal: Construct a DFA accepting all strings over $\{0, 1\}$ with an **odd number of 1s**.

Thought Process: We don't need to remember the entire string. We only need to remember whether the number of 1s seen so far is even or odd.



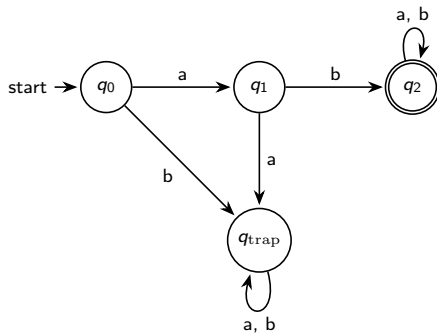
- **States:** $Q = \{q_{\text{even}}, q_{\text{odd}}\}$.
- **Start state:** q_{even} (zero 1s is even).
- **Accepting states:** $F = \{q_{\text{odd}}\}$.

Designing DFAs: Case Study 2 & Trap States

Goal: Construct a DFA over $\{a, b\}$ accepting strings beginning with the prefix ab .

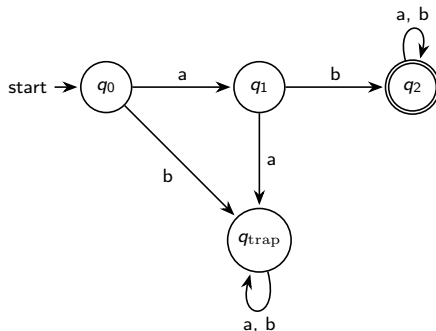
Designing DFAs: Case Study 2 & Trap States

Goal: Construct a DFA over $\{a, b\}$ accepting strings beginning with the prefix ab .



Designing DFAs: Case Study 2 & Trap States

Goal: Construct a DFA over $\{a, b\}$ accepting strings beginning with the prefix ab .



Teacher's Corner: The Trap State

A DFA may not simply omit a transition: δ must be total. Once the required prefix becomes impossible, the machine enters a nonaccepting **trap** (or sink) state and remains there.

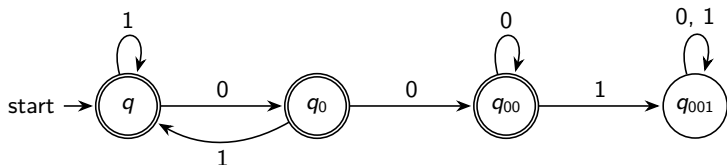
Designing DFAs: Case Study 3

Goal: Construct a DFA over $\{0, 1\}$ recognizing strings that do not contain the substring 001.

Designing DFAs: Case Study 3

Goal: Construct a DFA over $\{0, 1\}$ recognizing strings that do not contain the substring 001.

Strategy: Track the longest suffix of the input that is also a prefix of 001. Once 001 occurs, enter a rejecting trap state. Equivalently, one may complement the complete DFA for strings containing 001.



The states q , q_0 , q_{00} are accepting because the forbidden pattern has not yet appeared. State q_{001} is a rejecting trap state.

Part 3: Introduction to Nondeterminism

Self-Study Header: The Power of Choice

Nondeterminism is a generalization of determinism, so every deterministic finite automaton is automatically a nondeterministic finite automaton.

Part 3: Introduction to Nondeterminism

Self-Study Header: The Power of Choice

Nondeterminism is a generalization of determinism, so every deterministic finite automaton is automatically a nondeterministic finite automaton.

- **Branching model:** We may imagine the machine exploring all permitted paths; this is a mathematical model, not literal hardware duplication.
- **Acceptance:** The machine accepts if *at least one* branch reaches an accept state at the end of the input.
- **Empty Transitions:** An NFA can feature arrows labeled with ε (or λ), allowing it to branch into multiple states without consuming an input symbol.

Formal Definition of an NFA

Here we use the common ε -NFA model. It is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ whose transition function may include ε -moves.

Formal Definition of an NFA

Here we use the common ε -NFA model. It is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ whose transition function may include ε -moves.

The Transition Function δ

Let $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$. The transition function takes a state and an input symbol or the empty string and produces the **set** of possible next states.

$$\delta : Q \times \Sigma_\varepsilon \longrightarrow \mathcal{P}(Q).$$

Formal Definition of an NFA

Here we use the common ε -NFA model. It is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ whose transition function may include ε -moves.

The Transition Function δ

Let $\Sigma_\varepsilon = \Sigma \cup \{\varepsilon\}$. The transition function takes a state and an input symbol or the empty string and produces the **set** of possible next states.

$$\delta : Q \times \Sigma_\varepsilon \longrightarrow \mathcal{P}(Q).$$

Here, $\mathcal{P}(Q)$ (or 2^Q) is the power set of Q . Since the output is a set of states:

- It can return one state: $\{q_1\}$.
- It can return several states: $\{q_1, q_2\}$.
- It can return $\emptyset$, meaning no path continues by that move.

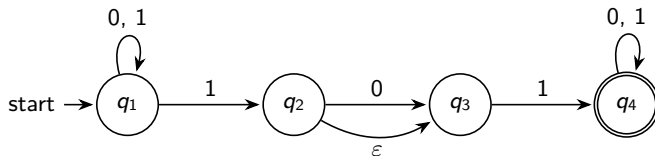
A string is accepted when at least one path consumes the entire string and finishes in a state of F .

NFA Example

Goal: Accept strings containing either 101 or 11 as a substring.

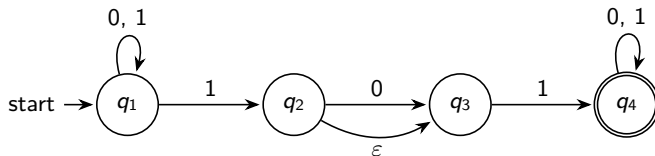
NFA Example

Goal: Accept strings containing either 101 or 11 as a substring.



NFA Example

Goal: Accept strings containing either 101 or 11 as a substring.



Why is this an NFA?

- 1 From q_1 , reading 1 may remain in q_1 or move to q_2 .
- 2 The path $q_2 \xrightarrow{0} q_3 \xrightarrow{1} q_4$ completes the substring 101.
- 3 Alternatively, $q_2 \xrightarrow{\varepsilon} q_3$ consumes no symbol, so the next 1 completes the substring 11.

Part 4: Equivalence Theorem

Theorem: Every nondeterministic finite automaton has an equivalent deterministic finite automaton.

Part 4: Equivalence Theorem

Theorem: Every nondeterministic finite automaton has an equivalent deterministic finite automaton.

This is a surprising but highly useful result. It tells us that adding nondeterminism *does not increase* the computational power of a finite automaton—they both recognize exactly the class of regular languages.

Part 4: Equivalence Theorem

Theorem: Every nondeterministic finite automaton has an equivalent deterministic finite automaton.

This is a surprising but highly useful result. It tells us that adding nondeterminism *does not increase* the computational power of a finite automaton—they both recognize exactly the class of regular languages.

Proof Idea: The Subset Construction Algorithm

If the NFA has k states, a DFA state records the set of NFA states reachable after the prefix read so far. At most 2^k subsets exist, though usually only some are reachable.

The Subset Construction Algorithm

Let $N = (Q, \Sigma, \delta_N, q_0, F_N)$ be an NFA. We build an equivalent DFA $M = (Q', \Sigma, \delta_M, q'_0, F')$.

The Subset Construction Algorithm

Let $N = (Q, \Sigma, \delta_N, q_0, F_N)$ be an NFA. We build an equivalent DFA $M = (Q', \Sigma, \delta_M, q'_0, F')$.

- **States:** Begin with $Q' = \mathcal{P}(Q)$ conceptually, then retain only subsets reachable from the start subset.

The Subset Construction Algorithm

Let $N = (Q, \Sigma, \delta_N, q_0, F_N)$ be an NFA. We build an equivalent DFA $M = (Q', \Sigma, \delta_M, q'_0, F')$.

- **States:** Begin with $Q' = \mathcal{P}(Q)$ conceptually, then retain only subsets reachable from the start subset.
- **ϵ -closure:** $E(R)$ contains the states reachable from R using zero or more ϵ -moves.

The Subset Construction Algorithm

Let $N = (Q, \Sigma, \delta_N, q_0, F_N)$ be an NFA. We build an equivalent DFA $M = (Q', \Sigma, \delta_M, q'_0, F')$.

- **States:** Begin with $Q' = \mathcal{P}(Q)$ conceptually, then retain only subsets reachable from the start subset.
- **ε -closure:** $E(R)$ contains the states reachable from R using zero or more ε -moves.
- **Start State (q'_0):** $q'_0 = E(\{q_0\})$. We begin by calculating the ε -closure of the NFA's start state.

The Subset Construction Algorithm

Let $N = (Q, \Sigma, \delta_N, q_0, F_N)$ be an NFA. We build an equivalent DFA $M = (Q', \Sigma, \delta_M, q'_0, F')$.

- **States:** Begin with $Q' = \mathcal{P}(Q)$ conceptually, then retain only subsets reachable from the start subset.
- **ε -closure:** $E(R)$ contains the states reachable from R using zero or more ε -moves.
- **Start State (q'_0):** $q'_0 = E(\{q_0\})$. We begin by calculating the ε -closure of the NFA's start state.
- **Transition function:** For $R \in Q'$ and $a \in \Sigma$,

$$\delta_M(R, a) = E\left(\bigcup_{r \in R} \delta_N(r, a)\right).$$

The Subset Construction Algorithm

Let $N = (Q, \Sigma, \delta_N, q_0, F_N)$ be an NFA. We build an equivalent DFA $M = (Q', \Sigma, \delta_M, q'_0, F')$.

- **States:** Begin with $Q' = \mathcal{P}(Q)$ conceptually, then retain only subsets reachable from the start subset.
- **ε -closure:** $E(R)$ contains the states reachable from R using zero or more ε -moves.
- **Start State (q'_0):** $q'_0 = E(\{q_0\})$. We begin by calculating the ε -closure of the NFA's start state.
- **Transition function:** For $R \in Q'$ and $a \in \Sigma$,

$$\delta_M(R, a) = E\left(\bigcup_{r \in R} \delta_N(r, a)\right).$$

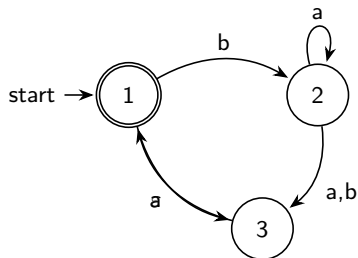
- **Accepting states:** $F' = \{R \in Q' : R \cap F_N \neq \emptyset\}$.

Correctness Invariant

After reading w , the DFA is in exactly the set of NFA states reachable from q_0 by paths whose non- ε labels spell w .

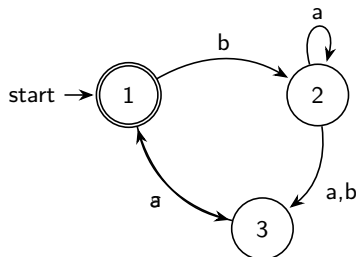
Worked Example: NFA to DFA

Consider the following NFA N_4 (states 1, 2, 3):



Worked Example: NFA to DFA

Consider the following NFA N_4 (states 1, 2, 3):



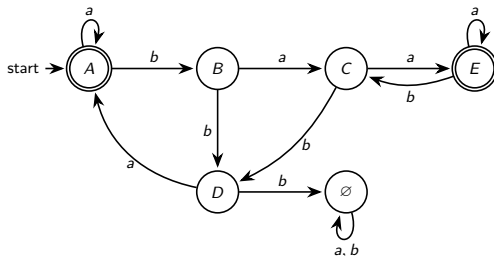
Start the subset construction:

- **Start State:** $E(\{1\}) = \{1, 3\}$ (since $1 \xrightarrow{a} 3$).
- **From $\{1, 3\}$ on a :** $\delta_N(1, a) = \emptyset$, $\delta_N(3, a) = \{1\}$. Closure $E(\{1\}) = \{1, 3\}$. So, $\delta_M(\{1, 3\}, a) = \{1, 3\}$.
- **From $\{1, 3\}$ on b :** $\delta_N(1, b) = \{2\}$, $\delta_N(3, b) = \emptyset$. Closure $E(\{2\}) = \{2\}$. So, $\delta_M(\{1, 3\}, b) = \{2\}$.

Worked Example: NFA to DFA (Completed)

Continuing until no new subset appears gives the following six reachable DFA states.

DFA state	a	b	accepting?
$A = \{1, 3\}$	A	B	✓
$B = \{2\}$	C	D	
$C = \{2, 3\}$	E	D	
$D = \{3\}$	A	$\emptyset$	
$E = \{1, 2, 3\}$	E	C	✓
$\emptyset$	$\emptyset$	$\emptyset$	



Only six of the $2^3 = 8$ possible subsets are reachable.

Part 5: Motivation for State Reduction

Self-Study Header: Finding the Smallest Equivalent DFA

For a given regular language, there are infinitely many DFAs that recognize it. For practical applications (like compiling and text searching), we want the **minimal** DFA—the one with the fewest possible states.

Part 5: Motivation for State Reduction

Self-Study Header: Finding the Smallest Equivalent DFA

For a given regular language, there are infinitely many DFAs that recognize it. For practical applications (like compiling and text searching), we want the **minimal** DFA—the one with the fewest possible states.

Indistinguishable States

States p and q are **equivalent** (indistinguishable), written $p \equiv q$, if for every suffix $w \in \Sigma^*$,

$$\delta^*(p, w) \in F \iff \delta^*(q, w) \in F.$$

Otherwise they are distinguishable; any witnessing w is a **distinguishing string**.

Part 5: Motivation for State Reduction

Self-Study Header: Finding the Smallest Equivalent DFA

For a given regular language, there are infinitely many DFAs that recognize it. For practical applications (like compiling and text searching), we want the **minimal** DFA—the one with the fewest possible states.

Indistinguishable States

States p and q are **equivalent** (indistinguishable), written $p \equiv q$, if for every suffix $w \in \Sigma^*$,

$$\delta^*(p, w) \in F \iff \delta^*(q, w) \in F.$$

Otherwise they are distinguishable; any witnessing w is a **distinguishing string**.

Equivalent reachable states represent the same future behavior and may be merged without changing the recognized language.

The Table-Filling Algorithm

The table-filling algorithm finds every distinguishable pair of states.

- 1 **Initialize:** Remove all unreachable states.

The Table-Filling Algorithm

The table-filling algorithm finds every distinguishable pair of states.

- ➊ **Initialize:** Remove all unreachable states.
- ➋ **Initialize the table:** List every unordered pair $\{p, q\}$ and mark pairs with exactly one accepting state.

The Table-Filling Algorithm

The table-filling algorithm finds every distinguishable pair of states.

- ① **Initialize:** Remove all unreachable states.
- ② **Initialize the table:** List every unordered pair $\{p, q\}$ and mark pairs with exactly one accepting state.
- ③ **Iterate:** For every unmarked pair (p, q) and every symbol $a \in \Sigma$:
 - Compute $r = \delta(p, a)$ and $s = \delta(q, a)$.
 - If $\{r, s\}$ is marked, mark $\{p, q\}$.

The Table-Filling Algorithm

The table-filling algorithm finds every distinguishable pair of states.

- ➊ **Initialize:** Remove all unreachable states.
- ➋ **Initialize the table:** List every unordered pair $\{p, q\}$ and mark pairs with exactly one accepting state.
- ➌ **Iterate:** For every unmarked pair (p, q) and every symbol $a \in \Sigma$:
 - Compute $r = \delta(p, a)$ and $s = \delta(q, a)$.
 - If $\{r, s\}$ is marked, mark $\{p, q\}$.
- ➍ **Terminate:** Repeat step 3 until a full pass makes no new marks.
- ➎ **Merge:** The remaining unmarked pairs are equivalent; collapse each equivalence class to one state.

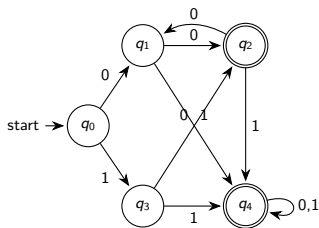
Minimal-DFA Theorem

After unreachable states are removed and equivalent states are merged, the resulting DFA is minimal and is unique up to renaming states.

Minimization Example

Consider a DFA with equivalent states.

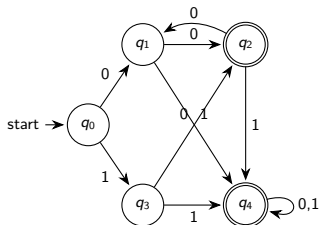
Original DFA



Minimization Example

Consider a DFA with equivalent states.

Original DFA

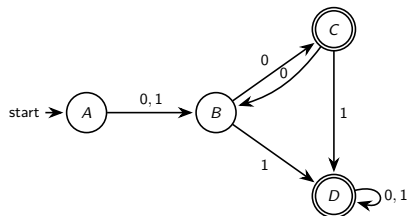


Equivalence classes

Only $q_1 \equiv q_3$. For example, suffix 0 distinguishes q_0 from q_1 , and also distinguishes q_2 from q_4 . Thus the classes are

$$A = \{q_0\}, \quad B = \{q_1, q_3\},$$

$$C = \{q_2\}, \quad D = \{q_4\}.$$



Checkpoint Exercises

- ① **DFA Design:** Draw the state diagram of a DFA over the alphabet $\Sigma = \{0, 1\}$ that recognizes the language: $\{w \mid w \text{ contains at least two 0s}\}$.
- ② **NFA Trace:** Consider an NFA with states $\{q_0, q_1, q_2\}$. The start state is q_0 .
- $\delta(q_0, \varepsilon) = \{q_1\}$
 - $\delta(q_1, \varepsilon) = \{q_2\}$
 - $\delta(q_0, a) = \{q_0\}$

What is the ε -closure of the start state q_0 ? That is, find $E(\{q_0\})$.

- ③ **Minimization Logic:** A complete DFA has exactly two reachable states, one accepting and one nonaccepting. Can these states be merged? Give a distinguishing string.

Hints for Checkpoint Exercises

Hint for Exercise 1 (DFA Design):

- Use states for having seen no 0s, exactly one 0, and at least two 0s.
- The last state is an accepting sink state.

Hint for Exercise 2 (ε -closure):

- A state is always in its own ε -closure.
- The closure is transitive! If you can reach q_1 from q_0 , and q_2 from q_1 via ε , you can reach q_2 from q_0 without consuming input.
- Therefore $E(\{q_0\}) = \{q_0, q_1, q_2\}$.

Hint for Exercise 3 (Minimization):

- The empty string ε distinguishes the two states: from one it is accepted immediately, and from the other it is rejected. Therefore the states cannot be merged.